Society Maintenance Management System (SMMS) — File Directory & Descriptions

This document provides a complete file-by-file inventory and descriptive guide of the SMMS codebase, covering all custom source code files across the root directory, backend API, frontend dashboard, Model Context Protocol (MCP) server, and telemetry services.

---

1. Root & Infrastructure Configuration

These files govern the local environment variables, container composition, routing, and automation pipelines.

* **[.env.example](file:///d:/projects/Society_mcp_server/.env.example)**: Environment variables template outlining database connections, API secrets, service ports, and optional OpenAI integrations.
* **[.gitignore](file:///d:/projects/Society_mcp_server/.gitignore)**: Directs Git to ignore transient directories such as `node_modules`, build outputs (`dist`), local environment overrides, and operating system metadata.
* **[docker-compose.yml](file:///d:/projects/Society_mcp_server/docker-compose.yml)**: Multi-container configuration orchestrating MongoDB, Express Backend, MCP SSE Server, React Frontend, Nginx Proxy, Prometheus, and Grafana.
* **[nginx.conf](file:///d:/projects/Society_mcp_server/nginx.conf)**: Proxy configuration mapping client request pathways (HTTP React assets to port 80, REST APIs to port 5000, and SSE messages to port 3001).
* **[.github/workflows/ci-cd.yml](file:///d:/projects/Society_mcp_server/.github/workflows/ci-cd.yml)**: GitHub Actions workflow configuration automating continuous integration, dependency checks, and builds.

---

2. Express Backend (`backend/`)

The backend is built with Express and Mongoose, handles auth, REST endpoints, and manages the AI integration.

Core Configuration & Boot
* **[backend/.dockerignore](file:///d:/projects/Society_mcp_server/backend/.dockerignore)**: Excludes local node modules and build outputs from the backend container compilation.
* **[backend/Dockerfile](file:///d:/projects/Society_mcp_server/backend/Dockerfile)**: Multi-stage build manifest compiling TypeScript source code and exposing the Express server on port 5000.
* **[backend/tsconfig.json](file:///d:/projects/Society_mcp_server/backend/tsconfig.json)**: TypeScript compiler options (module resolution, compile targets, and path mappings).
* **[backend/src/app.ts](file:///d:/projects/Society_mcp_server/backend/src/app.ts)**: Express application boots here. It registers routing tables,

global telemetry metrics, security middlewares, and error-handling utilities.
*
[backend/src/config/database.ts](file:///d:/projects/Society_mcp_server/backend/src/config/database.ts): Configures the Mongoose connection to the MongoDB cluster.

Middlewares
*
[backend/src/middlewares/auth.middleware.ts](file:///d:/projects/Society_mcp_server/backend/src/middlewares/auth.middleware.ts): Decodes JWT headers and restricts write access using role-based controls.
*
[backend/src/middlewares/error.middleware.ts](file:///d:/projects/Society_mcp_server/backend/src/middlewares/error.middleware.ts): Centralized error handling catching validation failures, duplicate keys, database cast errors, and authorization expiries.
*
[backend/src/middlewares/logger.middleware.ts](file:///d:/projects/Society_mcp_server/backend/src/middlewares/logger.middleware.ts): Access logger intercepting HTTP calls to print endpoints, method verbs, and response metrics.
*
[backend/src/middlewares/metrics.middleware.ts](file:///d:/projects/Society_mcp_server/backend/src/middlewares/metrics.middleware.ts): Tracks request durations and rates, publishing metrics to Prometheus.
*
[backend/src/middlewares/validation.middleware.ts](file:///d:/projects/Society_mcp_server/backend/src/middlewares/validation.middleware.ts): Validates request payloads (body, query, parameters) against Zod schemas before running controller methods.

Models (Mongoose Schemas)
*
[backend/src/models/Asset.ts](file:///d:/projects/Society_mcp_server/backend/src/models/Asset.ts): Tracks physical equipment (elevators, water pumps, generators), maintenance schedules, and failure risk indexes.
*
[backend/src/models/AuditLog.ts](file:///d:/projects/Society_mcp_server/backend/src/models/AuditLog.ts): Records administrative activity history (e.g. resident creations, payment status reconciliations) for security audits.
*
[backend/src/models/Complaint.ts](file:///d:/projects/Society_mcp_server/backend/src/models/Complaint.ts): Tracks housing complaints, complete with status histories, staff assignments, and automated AI triage metadata (sentiment scores, priorities, confidence ratings).
*
[backend/src/models/MaintenanceBill.ts](file:///d:/projects/Society_mcp_server/backend/src/models/MaintenanceBill.ts): Monthly maintenance invoices tracking due amounts, late payment fees, and payment state flags.
*
[backend/src/models/MaintenancePayment.ts](file:///d:/projects/Society_mcp_server/backend/src/models/MaintenancePayment.ts): Records billing transactions, tracking methods (UPI, card, cash) and transaction verification states.

*
[backend/src/models/Notice.ts](file:///d:/projects/Society_mcp_server/backend/src/models/Notice.ts): Public announcement notices with localized translations (English, Hindi, Marathi) and expiration controls.
*
[backend/src/models/RefreshToken.ts](file:///d:/projects/Society_mcp_server/backend/src/models/RefreshToken.ts): Manages refresh tokens, tracking rotation counts and token reuse to secure active sessions.
*
[backend/src/models/Resident.ts](file:///d:/projects/Society_mcp_server/backend/src/models/Resident.ts): Resident profile links (block letter, unit number, emergency details, family lists, vehicle registration numbers).
*
[backend/src/models/Society.ts](file:///d:/projects/Society_mcp_server/backend/src/models/Society.ts): Housing society settings, including billing cycles, maintenance rates, and address details.
*
[backend/src/models/User.ts](file:///d:/projects/Society_mcp_server/backend/src/models/User.ts): Main authentication profile storing emails, roles (Admin, Tenant, Staff, Vendor), and hashed passwords (bcrypt).
*
[backend/src/models/Vendor.ts](file:///d:/projects/Society_mcp_server/backend/src/models/Vendor.ts): Vendor database profile managing service category contracts (plumber, security, elevator repair) and service ratings.

Controllers
*
[backend/src/controllers/auth.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/auth.controller.ts): Manages authentication flows (signup, logins, refresh tokens, logouts, profile queries) and contains the **auto-seeding engine** for new society registration profiles.
*
[backend/src/controllers/bill.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/bill.controller.ts): Administrative controls for generating society billing invoices and querying resident outstanding payments.
*
[backend/src/controllers/complaint.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/complaint.controller.ts): Handles ticket creations, AI triage calls, updating repair workflows, and assigning technicians.
*
[backend/src/controllers/notice.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/notice.controller.ts): Handles publishing and pinning notices, including automatic translations for Hindi and Marathi.
*
[backend/src/controllers/payment.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/payment.controller.ts): Records resident transactions, verifying payments and updating outstanding bills.
*
[backend/src/controllers/resident.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/resident.controller.ts): Resident roster management (adds, updates, removals, and parking slot assignments).

*
[backend/src/controllers/vendor.controller.ts](file:///d:/projects/Society_mcp_server/backend/src/controllers/vendor.controller.ts): Manages contractor assignments, contracts, updates, and resident ratings.

Routes
*
[backend/src/routes/index.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/index.ts): Master routes router organizing endpoints under `/api/v1` namespace.
*
[backend/src/routes/auth.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/auth.routes.ts): Mounts register, login, profile, and logout endpoints.
*
[backend/src/routes/bill.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/bill.routes.ts): Routes for managing maintenance bills.
*
[backend/src/routes/complaint.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/complaint.routes.ts): Routes for filing and resolving complaints.
*
[backend/src/routes/notice.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/notice.routes.ts): Routes for managing announcements.
*
[backend/src/routes/payment.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/payment.routes.ts): Routes for recording payments.
*
[backend/src/routes/resident.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/resident.routes.ts): Routes for managing resident details.
*
[backend/src/routes/vendor.routes.ts](file:///d:/projects/Society_mcp_server/backend/src/routes/vendor.routes.ts): Routes for managing vendors.

Services & Utils
*
[backend/src/services/ai.service.ts](file:///d:/projects/Society_mcp_server/backend/src/services/ai.service.ts): Contains the AI integrations for classifying complaints, translating notices, and predicting equipment failure risk.
*
[backend/src/utils/appError.ts](file:///d:/projects/Society_mcp_server/backend/src/utils/appError.ts): Defines a custom error class for handling operational errors with HTTP status codes.
*
[backend/src/utils/logger.ts](file:///d:/projects/Society_mcp_server/backend/src/utils/logger.ts): Winston logger configuration for saving debug and error logs to file targets and the console.
*
[backend/src/utils/metrics.ts](file:///d:/projects/Society_mcp_server/backend/src/utils/metrics.ts): Configures Prometheus client metrics (e.g. request counters, database connection status gauge).

Zod Validators
*
[backend/src/validators/auth.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/auth.validator.ts): Enforces validation rules for user register and login requests.
*
[backend/src/validators/bill.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/bill.validator.ts): Validates maintenance bill invoice creation payloads.
*
[backend/src/validators/complaint.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/complaint.validator.ts): Validates complaint ticket creation and status update payloads.
*
[backend/src/validators/notice.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/notice.validator.ts): Validates notice details and target audience parameters.
*
[backend/src/validators/payment.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/payment.validator.ts): Validates transaction and billing settlement fields.
*
[backend/src/validators/resident.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/resident.validator.ts): Validates resident registrations, flat mappings, and family lists.
*
[backend/src/validators/vendor.validator.ts](file:///d:/projects/Society_mcp_server/backend/src/validators/vendor.validator.ts): Validates vendor contract updates and rating request body payloads.

---

3. Frontend Client SPA (`frontend/`)

Built with Vite, React 18, TailwindCSS, Redux Toolkit, and React Query, the frontend is a dashboard for administrators and residents.

Core Settings & Entry
*
[frontend/.dockerignore](file:///d:/projects/Society_mcp_server/frontend/.dockerignore): Excludes local packages and development config files from the Docker build context.
*
[frontend/Dockerfile](file:///d:/projects/Society_mcp_server/frontend/Dockerfile): Multi-stage build file that compiles the React SPA and serves it via an Nginx container on port 80.
*
[frontend/nginx.conf](file:///d:/projects/Society_mcp_server/frontend/nginx.conf): Configures Nginx to route all requests back to `index.html` to support React Router SPA paths.
*
[frontend/index.html](file:///d:/projects/Society_mcp_server/frontend/index.html): Main HTML skeleton where Vite mounts the React application.
*
[frontend/postcss.config.js](file:///d:/projects/Society_mcp_server/frontend/postcss.config.js): Configures PostCSS dependencies for compiling Tailwind styles.

*
[frontend/tailwind.config.js](file:///d:/projects/Society_mcp_server/frontend/tailwind.config.js): Theme configuration file defining the custom dark glassmorphic color palette, shadows, and animations.
*
[frontend/vite.config.ts](file:///d:/projects/Society_mcp_server/frontend/vite.config.ts): Configures Vite plugins (such as React and TS paths) and defines dev server configurations.
*
[frontend/tsconfig.json](file:///d:/projects/Society_mcp_server/frontend/tsconfig.json): TypeScript compiler configurations for the Vite React workspace.
*
[frontend/src/main.tsx](file:///d:/projects/Society_mcp_server/frontend/src/main.tsx): Entry point that wraps the app in Redux Store and React Query context providers.
*
[frontend/src/index.css](file:///d:/projects/Society_mcp_server/frontend/src/index.css): Global CSS stylesheet defining design tokens, base styles, and utility classes.
*
[frontend/src/App.tsx](file:///d:/projects/Society_mcp_server/frontend/src/App.tsx): Routes manager mapping URLs to dashboard page views.

Integration Client
*
[frontend/src/api/axiosClient.ts](file:///d:/projects/Society_mcp_server/frontend/src/api/axiosClient.ts): Axios client instance configured with interceptors to handle automatic silent token refreshes and request queues.

Shared Layout & Guards
*
[frontend/src/components/Layout.tsx](file:///d:/projects/Society_mcp_server/frontend/src/components/Layout.tsx): Persistent dashboard wrapper containing the sidebar navigation, user headers, and health checks.
*
[frontend/src/components/ProtectedRoute.tsx](file:///d:/projects/Society_mcp_server/frontend/src/components/ProtectedRoute.tsx): Authentication guard that checks the user's role and redirects unauthenticated requests to the login screen.

Redux Store Config
*
[frontend/src/store/index.ts](file:///d:/projects/Society_mcp_server/frontend/src/store/index.ts): Global Redux store configuration.
*
[frontend/src/store/slices/authSlice.ts](file:///d:/projects/Society_mcp_server/frontend/src/store/slices/authSlice.ts): Slice managing active user state, JWT access tokens, and persistence keys.

Component Logic & Features
*
[frontend/src/features/auth/Login.tsx](file:///d:/projects/Society_mcp_server/frontend/src/features/auth/Login.tsx): Glassmorphic user login component containing authentication forms.
*
**[frontend/src/features/complaints/ComplaintList.tsx](file:///d:/project

s/Society_mcp_server/frontend/src/features/complaints/ComplaintList.tsx)**: Interactive table component displaying filed complaint tickets and status updates.
*
[frontend/src/features/dashboard/Dashboard.tsx](file:///d:/projects/Society_mcp_server/frontend/src/features/dashboard/Dashboard.tsx): Dashboard home screen component visualizing recent notice alerts, payment histories, and metrics.

Pages Wrapper
*
[frontend/src/pages/Login.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Login.tsx): Wrapper view page rendering the login form components.
*
[frontend/src/pages/Dashboard.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Dashboard.tsx): Page view orchestrating dashboard layouts and administrative metrics queries.
*
[frontend/src/pages/Complaints.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Complaints.tsx): Page view containing complaint forms, priority categorization lists, and technician assign workflows.
*
[frontend/src/pages/Notices.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Notices.tsx): Page view displaying active notices and announcements with multi-language toggle selectors.
*
[frontend/src/pages/Payments.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Payments.tsx): Page view for residents to pay outstanding dues and admins to reconcile ledger details.
*
[frontend/src/pages/Residents.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Residents.tsx): Page view for admins to view and manage the resident directory and flat assignments.
*
[frontend/src/pages/Reports.tsx](file:///d:/projects/Society_mcp_server/frontend/src/pages/Reports.tsx): Page view for generating and exporting PDF/CSV reports (collections summaries, occupancy tables, etc.).

---

4. MCP Server (`mcp-server/`)

The Model Context Protocol Server exposes database resources, triggers administrative tools, and integrates with AI agents.

Server Config & Boot
*   **[mcp-server/.dockerignore](file:///d:/projects/Society_mcp_server/mcp-server/.dockerignore)**: Excludes local builds and node modules from the MCP container compilation context.
*   **[mcp-server/Dockerfile](file:///d:/projects/Society_mcp_server/mcp-server/Dockerfile)**: Compiles the TypeScript code and boots the SSE Express app on port 3001.
*   **[mcp-server/tsconfig.json](file:///d:/projects/Society_mcp_server/mcp-

server/tsconfig.json)**: TypeScript compiler settings configured for Node.js modules and output formatting.
*   **[mcp-server/src/index.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/index.ts)**: Configures the Express server to support SSE transport connection handshakes (`GET /sse`, `POST /messages`), registers available tools, resources, and prompt templates.

Client Orchestration
*   **[mcp-server/src/client/openai-orchestrator.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/client/openai-orchestrator.ts)**: Agentic OpenAI test client that executes the reasoning loop, translates schemas, and processes natural language instructions like *"Who has not paid?"*. Contains a **mock simulation backup mode** to test execution flows without an active OpenAI API key.

Clean Architecture Domain Layer
*   **[mcp-server/src/domain/repositories/ComplaintRepository.interface.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/domain/repositories/ComplaintRepository.interface.ts)**: Domain interface contract for query interactions with complaint tickets.
*   **[mcp-server/src/domain/repositories/NoticeRepository.interface.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/domain/repositories/NoticeRepository.interface.ts)**: Domain interface contract for notice bulletin board interactions.
*   **[mcp-server/src/domain/repositories/PaymentRepository.interface.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/domain/repositories/PaymentRepository.interface.ts)**: Domain interface contract for payment transactions and outstanding invoices tracking.
*   **[mcp-server/src/domain/repositories/ResidentRepository.interface.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/domain/repositories/ResidentRepository.interface.ts)**: Domain interface contract for resident directory rosters.
*   **[mcp-server/src/domain/repositories/SocietyRepository.interface.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/domain/repositories/SocietyRepository.interface.ts)**: Domain interface contract for managing housing society details.
*   **[mcp-server/src/domain/repositories/UserRepository.interface.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/domain/repositories/UserRepository.interface.ts)**: Domain interface contract for user credentials lookup operations.

Clean Architecture Infrastructure Layer
*   **[mcp-server/src/infrastructure/repositories/ComplaintRepository.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/infrastructure/repositories/ComplaintRepository.ts)**: Mongoose implementation querying the MongoDB collections for housing complaints.

*   **[mcp-server/src/infrastructure/repositories/NoticeRepository.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/infrastructure/repositories/NoticeRepository.ts)**: Mongoose implementation querying notice bulletins.
*   **[mcp-server/src/infrastructure/repositories/PaymentRepository.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/infrastructure/repositories/PaymentRepository.ts)**: Mongoose implementation querying billing details and transaction records.
*   **[mcp-server/src/infrastructure/repositories/ResidentRepository.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/infrastructure/repositories/ResidentRepository.ts)**: Mongoose implementation querying unit details and resident profiles.
*   **[mcp-server/src/infrastructure/repositories/SocietyRepository.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/infrastructure/repositories/SocietyRepository.ts)**: Mongoose implementation querying society configuration profiles.
*   **[mcp-server/src/infrastructure/repositories/UserRepository.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/infrastructure/repositories/UserRepository.ts)**: Mongoose implementation querying user accounts.

MCP Models
*   **[mcp-server/src/models/Complaint.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/Complaint.ts)**: Database schema blueprint for complaint tickets.
*   **[mcp-server/src/models/MaintenanceBill.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/MaintenanceBill.ts)**: Database schema blueprint for maintenance bills.
*   **[mcp-server/src/models/MaintenancePayment.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/MaintenancePayment.ts)**: Database schema blueprint for payments.
*   **[mcp-server/src/models/Notice.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/Notice.ts)**: Database schema blueprint for notices.
*   **[mcp-server/src/models/Resident.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/Resident.ts)**: Database schema blueprint for resident profiles.
*   **[mcp-server/src/models/Society.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/Society.ts)**: Database schema blueprint for society profiles.
*   **[mcp-server/src/models/User.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/models/User.ts)**: Database schema blueprint for user accounts.

Schemas & Services
*   **[mcp-server/src/schemas/toolSchemas.ts](file:///d:/projects/Society_mcp_server

/mcp-server/src/schemas/toolSchemas.ts)**: Zod schemas validation file ensuring tool arguments comply with MCP specifications.
*   **[mcp-server/src/services/CreateNoticeService.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/services/CreateNoticeService.ts)**: Business logic to publish society notices.
*   **[mcp-server/src/services/GenerateReportService.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/services/GenerateReportService.ts)**: Logic to compile collection balances, tickets, and occupancy statistics.
*   **[mcp-server/src/services/GetPendingPaymentsService.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/services/GetPendingPaymentsService.ts)**: Logic to query unpaid invoices.
*   **[mcp-server/src/services/GetResidentService.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/services/GetResidentService.ts)**: Logic to find resident details by unit number.
*   **[mcp-server/src/services/RegisterComplaintService.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/services/RegisterComplaintService.ts)**: Logic to create complaint tickets and assign technician categories.
*   **[mcp-server/src/services/SendReminderService.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/services/SendReminderService.ts)**: Business logic to create payment reminders.

Executable Clients & Tests
*   **[mcp-server/src/test-client.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/test-client.ts)**: Connects directly via standard I/O (stdin/stdout) transport channels to verify tools and prompts locally.
*   **[mcp-server/src/test-sse-client.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/test-sse-client.ts)**: Simulates client-server handshakes over HTTP SSE to verify web connections.
*   **[mcp-server/src/tests/run-tests.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/run-tests.ts)**: Executable script running the system integration test suite.
*   **[mcp-server/src/tests/CreateNoticeService.test.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/CreateNoticeService.test.ts)**: Integration tests verifying notice creation and translation workflows.
*   **[mcp-server/src/tests/GenerateReportService.test.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/GenerateReportService.test.ts)**: Integration tests verifying financial report compilations.
*   **[mcp-server/src/tests/GetPendingPaymentsService.test.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/GetPendingPaymentsService.test.ts)**: Integration tests verifying pending invoice lists.
*   **[mcp-server/src/tests/GetResidentService.test.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/GetResidentService.test.ts)**: Integration tests verifying resident profiles.

*   **[mcp-server/src/tests/RegisterComplaintService.test.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/RegisterComplaintService.test.ts)**: Integration tests verifying AI ticket categorization.
*   **[mcp-server/src/tests/SendReminderService.test.ts](file:///d:/projects/Society_mcp_server/mcp-server/src/tests/SendReminderService.test.ts)**: Integration tests verifying payment alert messages.

---

5. Observability & Telemetry (`observability/`)

These configurations manage the collection and visualization of system metrics.

*   **[observability/prometheus/prometheus.yml](file:///d:/projects/Society_mcp_server/observability/prometheus/prometheus.yml)**: Configures Prometheus to scrape `/metrics` endpoints from the backend and MCP servers every 15 seconds.
*   **[observability/prometheus/alert.rules.yml](file:///d:/projects/Society_mcp_server/observability/prometheus/alert.rules.yml)**: Contains Prometheus alert rules (e.g. flagging high HTTP request failure rates or CPU usage spikes).
*   **[observability/grafana/provisioning/datasources/prometheus.yaml](file:///d:/projects/Society_mcp_server/observability/grafana/provisioning/datasources/prometheus.yaml)**: Configures Prometheus as the default datasource for Grafana.
*   **[observability/grafana/provisioning/dashboards/dashboard.yaml](file:///d:/projects/Society_mcp_server/observability/grafana/provisioning/dashboards/dashboard.yaml)**: Provisions dashboard dashboards folder target paths inside Grafana containers.
*   **[observability/grafana/dashboards/node-express.json](file:///d:/projects/Society_mcp_server/observability/grafana/dashboards/node-express.json)**: JSON dashboard configuration containing visualizations for monitoring request rates, error ratios, database states, and response latencies.